



JavaOneSM
Sun's 2000 Worldwide Java Developer Conference™

Future Directions of the JDBCTM API

Jon Ellis and Linda Ho
JDBC Development
Sun Microsystems, Inc.

John Goodson
Merant

Agenda

- JDBC™ 3.0 API
- RowSets
- Building the WebRowSet Demo



JDBC 3.0 API

- Introduction
- Prior releases
- Current developments

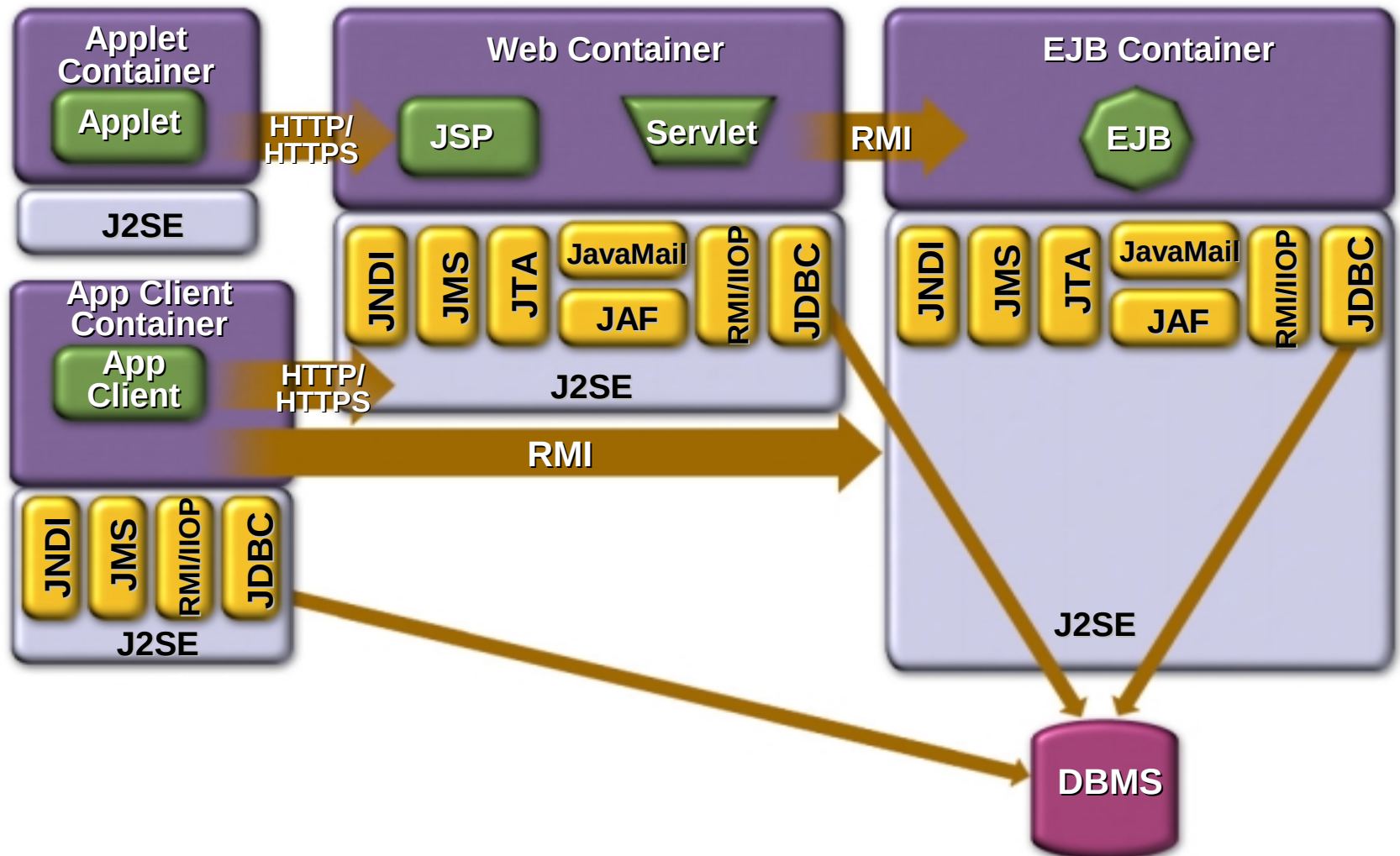


The JDBC API

- Programmatic access to relational data from the Java™ platform
- Connectivity to any data source with a corresponding JDBC API-based driver
- Connectivity to heterogeneous data sources



JDBC API in the Java Platform



Prior Releases

- JDBC 1.0 extension
 - Focus on providing a call-level interface to SQL
 - DriverManager, Connections, Statements, ResultSets
- JDBC 2.1 and 2.0 extension
 - Focus on portable enterprise applications
 - Scrollable cursors, batch updates
 - DataSources, connection pooling, distributed transactions
 - RowSets



The Next Release

- JDBC 3.0 API is currently being developed
- Using JCP (JSR 0054)
- Goals for JDBC 3.0 API
 - Reflect changes made to SQL 99 and database systems since JDBC 2.0 API
 - Keep pace with the Java platform
 - Enable plug-ability with emerging technologies for the Java platform (Connector architecture)
 - Targeted to be released with the J2SE™ 1.4 platform
 - Consolidation of specifications



Potential New Developments

- Connection pool configuration
- Reuse of prepared SQL with connection pools
- Updatable BLOB and CLOB
- UDT and UDF meta-data
- Describe input host variables
- SQL99 transaction constructs
- Bundling javax.sql into core



Conclusion

- The JDBC API is a mature technology
- Future work will exploit the extensibility already present





JavaOneSM
Sun's 2000 Worldwide Java Developer Conference™

RowSets: Portable Data Access in Enterprise Applications

RowSets

- RowSet interface
- Three implementations
- Extensibility



Motivation

- Tabular data prevalent
- Component model highly distributed
- Decouple tabular data from data source
- First class component

**RowSets are a
Tabular Data Datatype**



Background

- JDBC 2.0 API Optional Package
`javax.sql.RowSet`
- Extend the capabilities of
`java.sql.ResultSet`
 - Disconnected operations
 - Serializable
 - Extensible to non-tabular data sources
- JavaBeans™ architecture-based
Components
 - Properties
 - Event model



Properties of JavaBeansTM Technology-based Components

- Connection properties
 - URL/datasource
 - Username/password
 - Query

```
CachedRowSet crs = new  
    CachedRowSet();  
crs.setDataSource("jdbc/db");  
crs.setUserName("bob");  
crs.setCommand("select ... ");  
crs.execute();
```



JavaBeans Components Events

- Components register to be notified of events
 - cursorMoved
 - rowChanged
 - rowSetChanged

```
crs.addRowSetListener(pieChart);
```



Disconnected Operation

```
//create a resultSet to populate  
//RowSet  
stmt = conn.createStatement();  
rs = stmt.executeQuery("select ...");  
CachedRowSet crs = new CachedRowSet();  
crs.populate(rs);
```

```
// close the connection  
conn.close();
```

```
// RowSet still contains values...  
crs.next();  
Object col1 = crs.getObject(1);
```



Serializable

- RowSets can be passed between distributed components within a tier
- RowSets can be passed between tiers—from server to client and back
- Serializability reduces the need to map to objects



Extensible

- Custom reader/writers
 - Use properties to establish connection
 - Read/write data from non-relational data sources
- Create RowSets independent of a data source



Implementations

- Connected RowSets
 - JDBCRowSet
- Disconnected RowSets
 - CachedRowSet
 - WebRowSet



Connected RowSet: JDBCRowSet

- Maintains connection
- Makes a JDBC API-based driver look like a JavaBeans component
- Useful for building applications with tools



Disconnected RowSets: CachedRowSet

- Rows are cached in memory
- Updatable
- Serializable
- Usage
 - Pass tabular data between components and tiers
 - Provide scrollable updatable cursor support



Synchronizing Changes

- RowSet updates need to be propagated back to the underlying data source

```
crs.next();
```

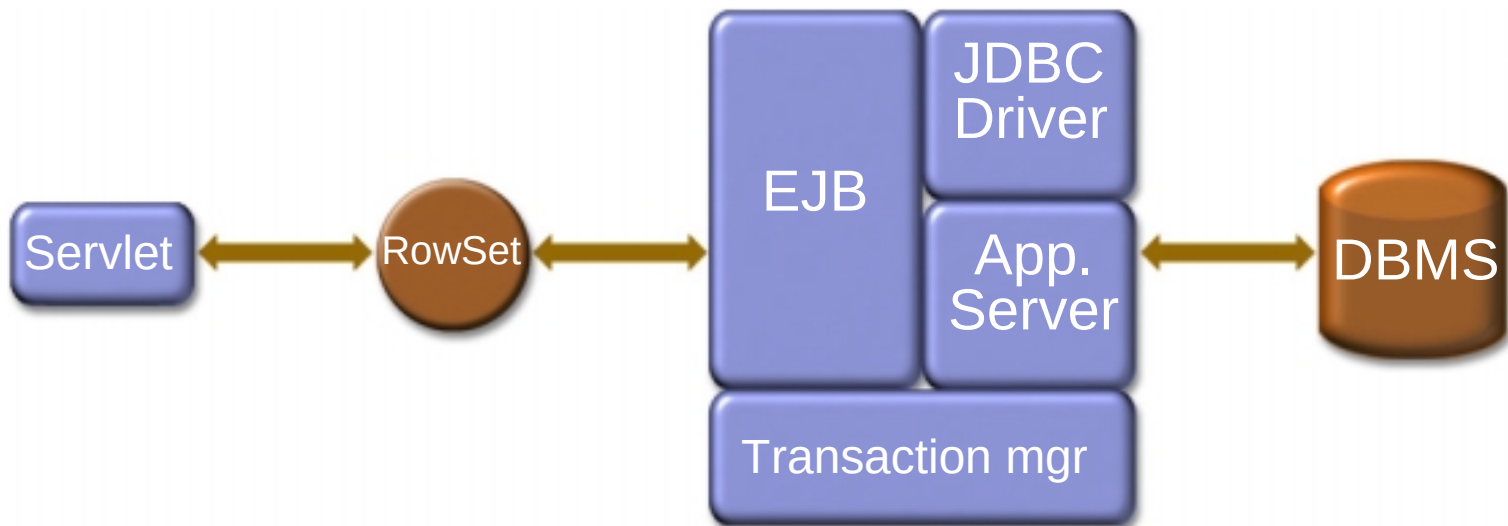
```
// update the value in RowSet  
crs.updateString(name, "Smith");  
crs.updateRow();
```

```
// update the underlying data source  
crs.acceptChanges();
```



Distributed Application

- Passed to/returned from EJB component



Disconnected RowSets: WebRowSets

- Based on CachedRowSet
- Adds support for serialization to XML

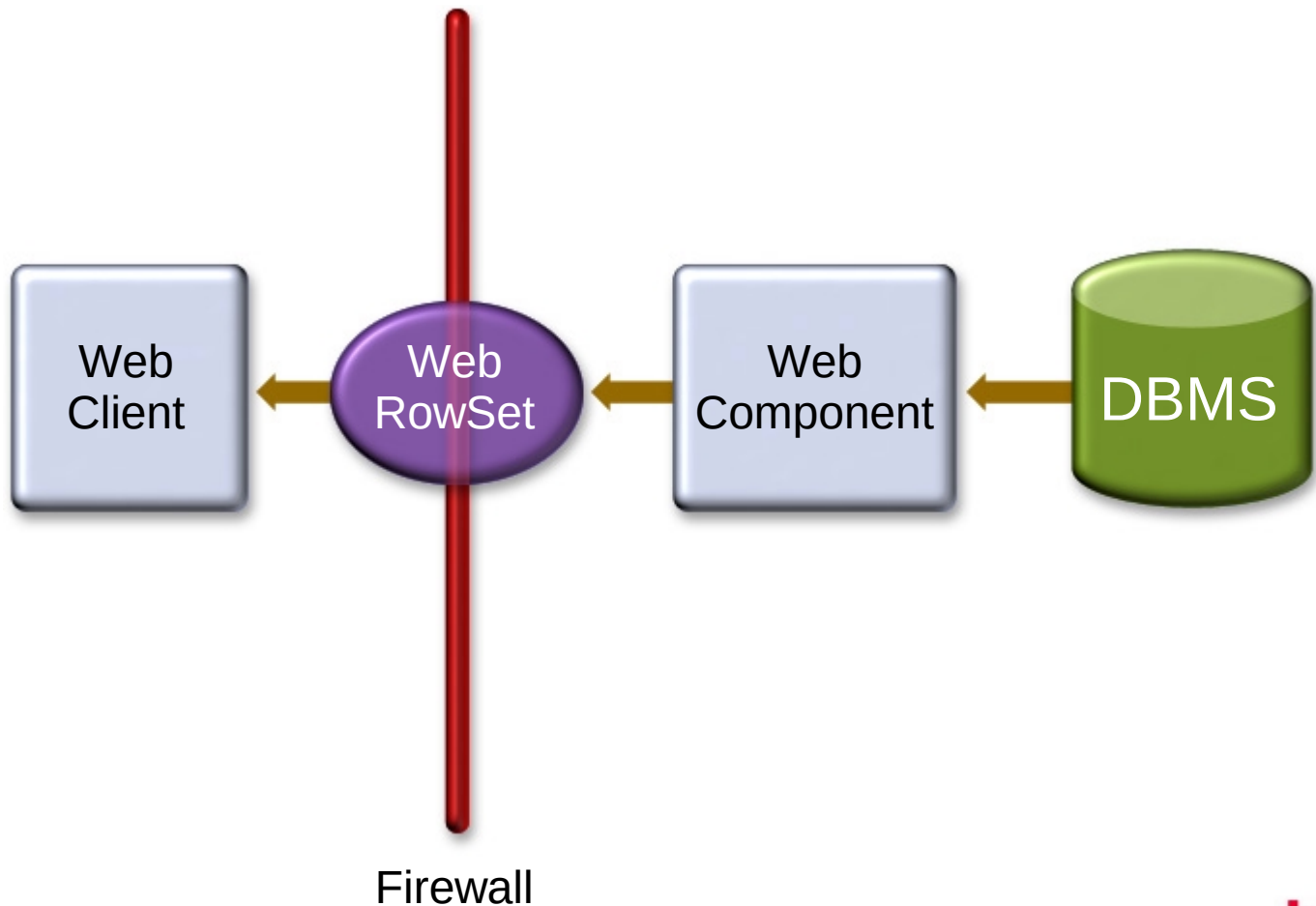
```
WebRowSet wrs = new WebRowSet();
```

```
// populate the RowSet  
wrs.populate(rs);
```

```
// write the RowSet out as XML  
wrs.writeXML(out);
```



WebRowSet Serialized to XML



XML Document

- Three main sections:
 - Properties
 - Metadata
 - Data
- Represents the complete state of the RowSet including changes made to the data: updates, inserts, deletes

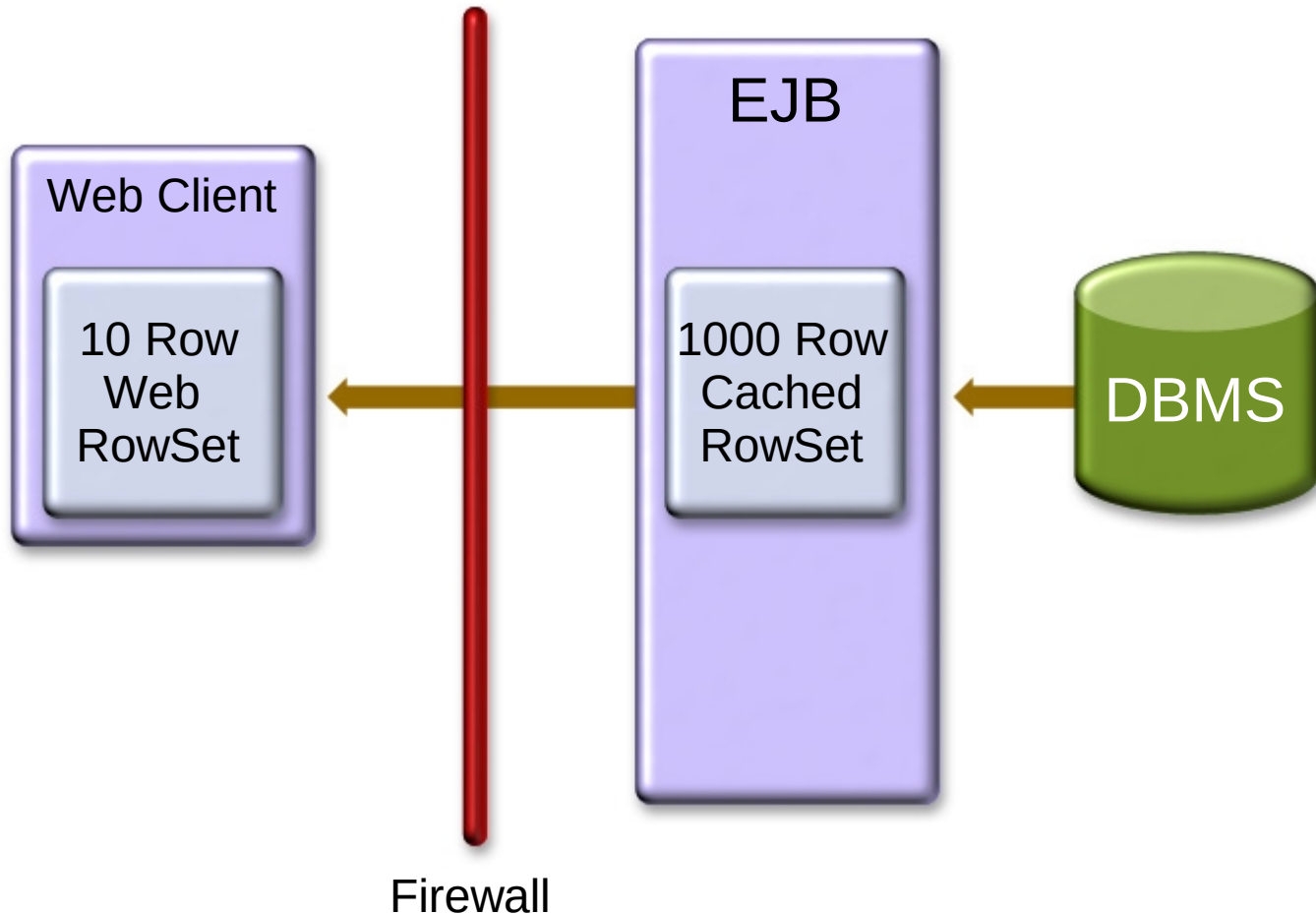
XML Document

```
<ResultSet>
  <properties>
    <username>"bert"</username>
    ...
  </properties>
  <meta-data>
    <column-count>2</column-count>
    ...
  </meta-data>
  <data>
    <row>
      <col>"Smith"</col>
    </row>
    ...
  </data>
</ResultSet>
```

Extensibility

- Not restricted to relational data sources
- Possible to have **custom readers** backed by EJB components
 - Windowing rows to a client
 - Maintain state and deliver rows in chunks

“Chunking” Rows



Custom Readers

- Supply meta-data information
- Insert data using ResultSet interface

```
CachedRowSet crs = new  
                                CachedRowSet();  
// setup meta data  
RowSetMetaData md =  
                                createMetaData();  
crs.setMetaData(metaData);  
crs.moveToInsertRow();  
crs.updateString(1, ...);  
...  
crs.insertRow();
```



Custom Writers

- Writers propagate changes back to underlying data source
- Writers use RowSetInternal interface to identify modified rows
- Optimistic concurrency control
 - Identify original row
 - Perform updates/delete
 - May fail if underlying data has changed



Conclusion

- Tabular data is everywhere in a distributed enterprise application environment
- RowSets ship tabular data **anywhere**



Resources

- JDBC API website at:
<http://java.sun.com/products/jdbc>
- RowSet implementations at:
<http://java.sun.com/jdc>
- Send feedback/ideas to
jdbc@eng.sun.com

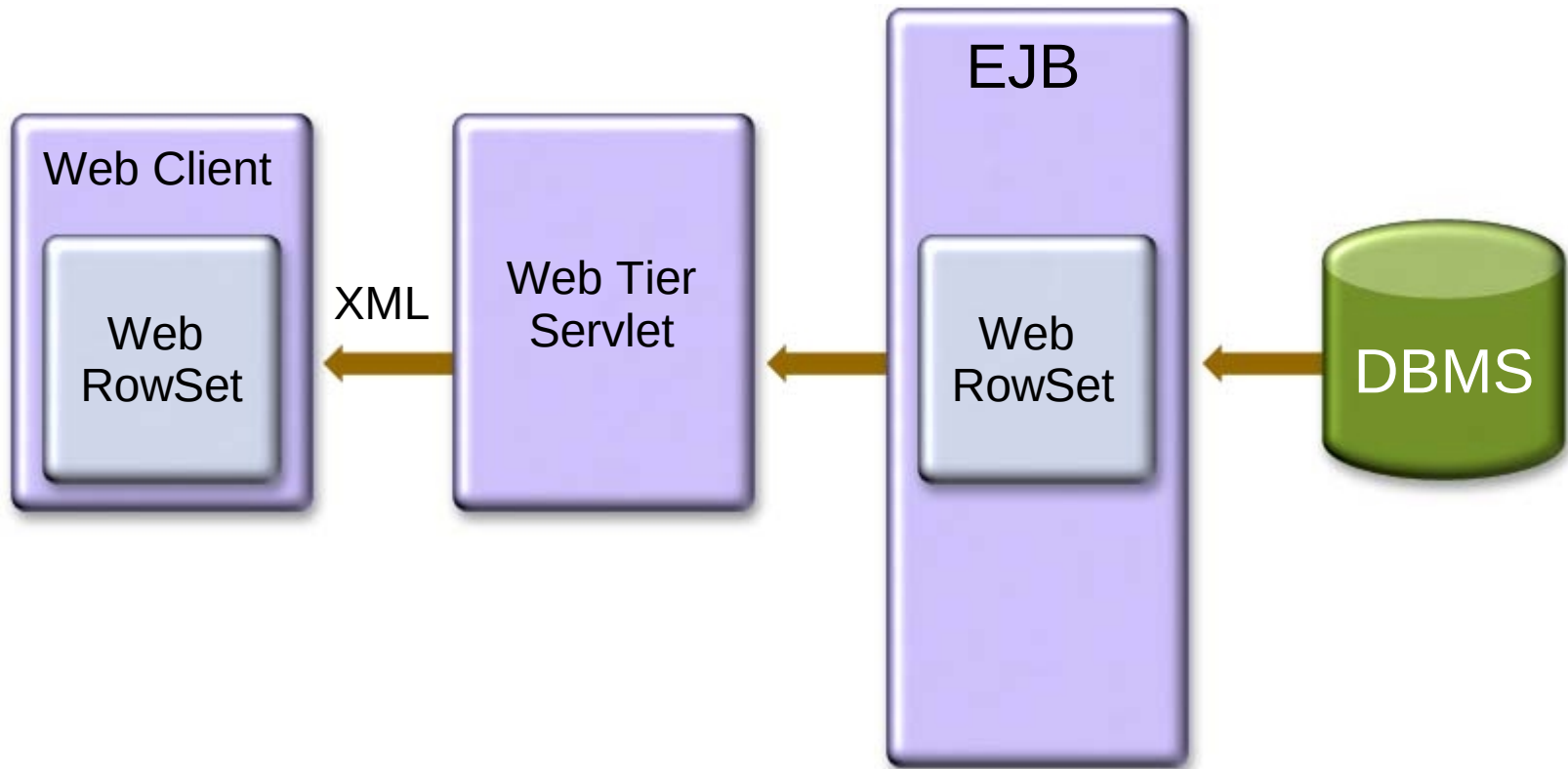




JavaOneSM
Sun's 2000 Worldwide Java Developer Conference™

Building the WebRowSet Demo

Structure of the Demo



EJB™ Technology-based Tier

- All data cached using a WebRowSet
- Contains business logic for returning weather data in logical chunks
- Custom Writer extracts only requested data



Web Tier

- Servlet
- Services requests from client (applet) to EJB tier
- Sends data in XML format to client

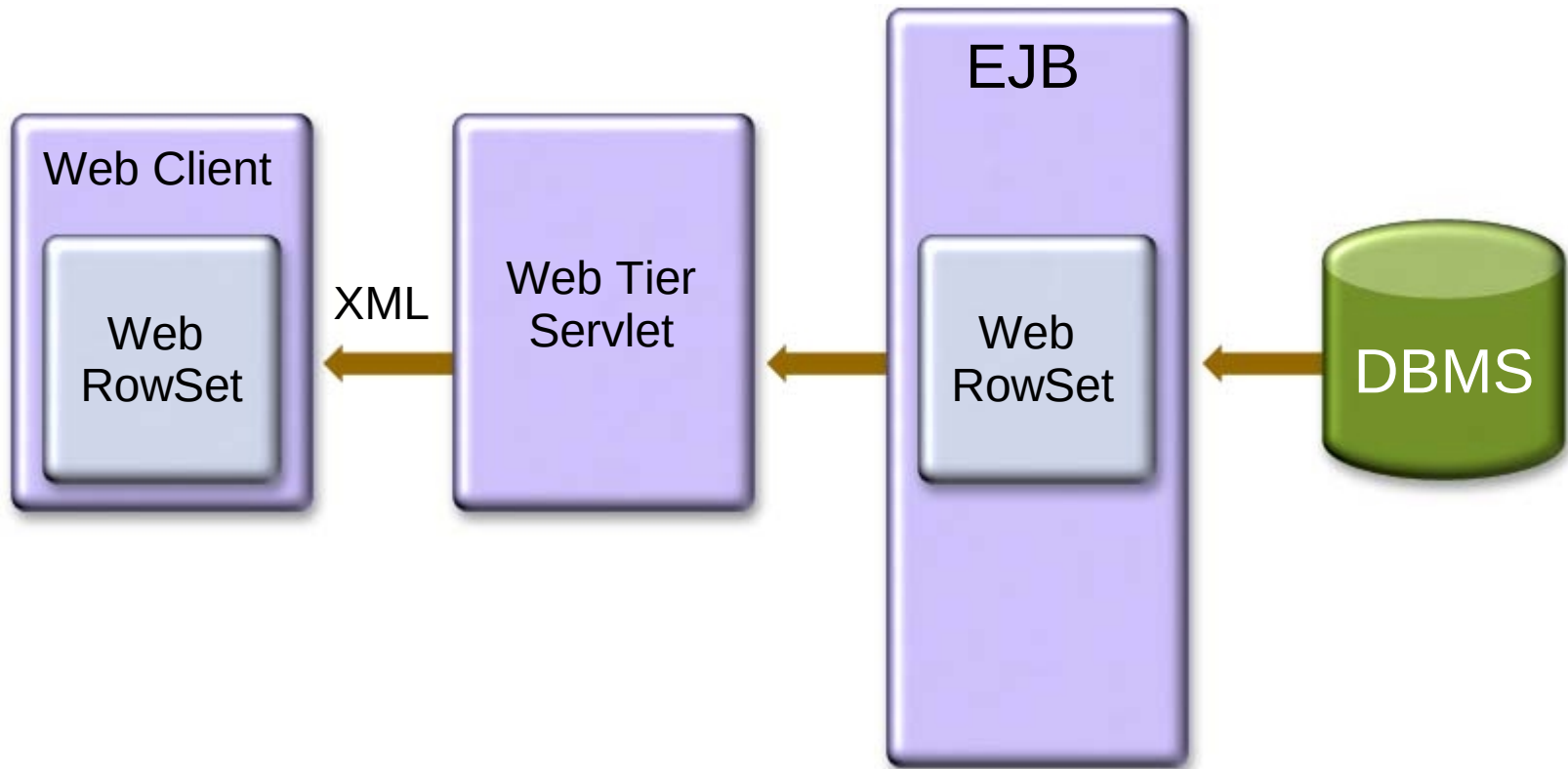


Client Tier

- Applet
- Maintains own WebRowSet for different user usage of the data (table vs. graph)
- Retrieves data from the Web Tier in XML format directly into the WebRowSet
- Services user requests through Web Tier



Structure of the Demo





JavaOneSM
Sun's 2000 Worldwide Java Developer Conference™

Q&A

Resources

- JDBC API website at:
<http://java.sun.com/products/jdbc>
- RowSet implementations at:
<http://java.sun.com/jdc>
- Send feedback/ideas to jdbc@eng.sun.com





JavaOneSM
Sun's 2000 Worldwide Java Developer Conference[®]